The Human CPU

CS 10 · Tue 3 Nov 2026 · P5+P6

Hardware booklet Parts C–D, pp. 26–36 · L16 of Autumn 2

P5 14:05–15:00 · P6 15:05–15:55

Before the lesson

- **The cards:** 14 sheets. Fold each along the dashed line into a tent, big label to the room. Six CPU roles, eight memory addresses.
- **Something to write on** for the PC, MAR, MDR and ACC: mini whiteboards if the room has them, otherwise scrap paper and a thick pen. Address 7 writes on its own card, in pencil.
- **A3 paper**, 7 sheets, for the P5 infographic (six pairs and a three).
- **Board:** an empty trace table with D.2's columns, ready for Run 2.
- **Short of students?** Staff the six CPU roles first. Any memory card can stand on a chair, holder side towards the CPU; the MAR walks over and reads it out.

P5 · Part C, inside the CPU, then the infographic

- **0–5** Retrieval, three on the board: what is cache for? · when is virtual memory used? · choose storage for a school's backups (choice · reason · consequence).
- **5–30** Part C, pp. 26–29: the CPU's job · addresses and data (C.3) · the components and their verbs (C.4) · the four registers, address or data (C.5, C.6).
- **30–50 Infographic in pairs, A3: "Who's who in the CPU".** Brief below, on the board.
- **50–55** Swap with the next pair. Each pair checks the other's verbs against p. 28 and tags against p. 29, and ticks every one that matches.

Infographic brief, for the board. Your poster shows:

1. Two boxes: the **CPU** and **main memory (RAM)**.
2. Inside the CPU: the **CU**, **ALU**, **cache** and **registers**, each with its verb.
3. The four registers, **PC, MAR, MDR, ACC**, each tagged *address* or *data*, with what it holds.
4. In memory: three locations from the memory map on p. 27, each with its address and its contents.

Check it. Every component has a verb, and every register has a tag.

P6 · Part D, the fetch-execute cycle, acted out

- **0–5** Cards out, Run 1 roles (cast list on p. 2). Read the key concept on p. 34 aloud: fetch, decode, execute. PC starts at 0; ACC starts empty.
- **5–20 Run 1, narrated:** the booklet's program. Say every cue for cycle 1 (p. 2), then let the CU lead from cycle 2 and prompt only when the room stalls. Leave the board table empty: D.2 asks them to recall this run.
- **20–32 Run 2, unnarrated:** everyone takes a different role. You fill in the board table as they go.
- **32–45** Seats: Practice D.1, D.2 (Run 1, from memory), D.3, pp. 35–36.
- **45–50** Exit: Practice D.5, why the PC goes up by 1 during every fetch [2]. *Answer:* so that the PC always holds the address of the next instruction (1); so the next cycle knows where to fetch from, and the instructions run in order (1).

Left for review study: pp. 18–25 (Part B practice) · p. 30 (C.7–C.9) · pp. 31–33 (Part C exam practice) · D.4 and D.6.

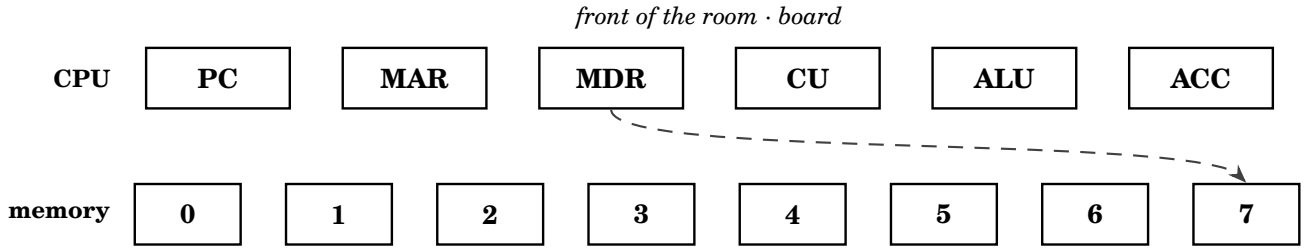
The cues for one cycle

Fetch	CU: "Fetch." → PC reads its address → MAR writes it and calls "Address *n*!" → location *n* reads its contents → MDR writes them → PC adds 1.
Decode	MDR reads the instruction to the CU → CU says what it means.
Execute	**LOAD *n***: CU "MAR, address *n*" → MAR calls it → location reads → MDR writes → CU "Accumulator, copy it" → ACC writes. **ADD *n***: as LOAD, as far as the MDR → CU "ALU, add" → ALU says the sum → ACC writes it. **STORE *n***: CU "MAR, address *n*. MDR, take the accumulator's number there" → MDR copies the ACC, walks it over → location writes it. **STOP**: CU "Stop. The program has ended."

Four points to make while it runs

- **The MAR and MDR work twice in a cycle:** in fetch, and again in execute for LOAD, ADD and STORE. That is why D.2's columns say *during fetch*. Say it the first time the MAR calls address 5 in Run 1.
- **The PC is one ahead of the MAR after every fetch** (the check on p. 35). In Run 2, ask the PC for its number before the CU decodes.
- **Reading memory copies; the location keeps its contents.** Address 5 is read twice in Run 2 and still says 12.
- **The CU gives the orders and the ALU does the sums** (p. 28's verbs).

Room layout



dashed: the MDR's walk for STORE 7

The two programs, and who plays what

CPU	Run 1	Run 2
PC		
MAR		
MDR		
CU		
ALU		
ACC		

Addr	Run 1	played by	Run 2	played by
0	LOAD 5		LOAD 6	
1	ADD 6		ADD 5	
2	STORE 7		ADD 5	
3	STOP		STORE 7	
4	–	*chair*	STOP	
5	12		12	
6	30		30	
7	→ 42		→ 54	*chair*

Run 1 is the memory map on pp. 27 and 35, the program D.2 asks about. Run 2 reads address 5 twice.

Run 1 trace = Practice D.2 answers

Cycle	MAR *during fetch*	MDR *during fetch*	PC *after fetch*	The CU decodes it as …	ACC *after execute*
1	0	LOAD 5	1	copy the number at address 5 into the accumulator	12
2	1	ADD 6	2	add the number at address 6 to the accumulator	42
3	2	STORE 7	3	copy the number in the accumulator into address 7	42
4	3	STOP	4	the program ends	42

D.3: address 7 holds **42**, which is **00101010** in 8-bit binary.

Run 2 trace, for the board

Cycle	MAR *during fetch*	MDR *during fetch*	PC *after fetch*	The CU decodes it as …	ACC *after execute*
1	0	LOAD 6	1	copy the number at address 6 into the accumulator	30
2	1	ADD 5	2	add the number at address 5 to the accumulator	42
3	2	ADD 5	3	add the number at address 5 to the accumulator	54
4	3	STORE 7	4	copy the number in the accumulator into address 7	54
5	4	STOP	5	the program ends	54

Address 7 ends on **54**, which is **00110110**.